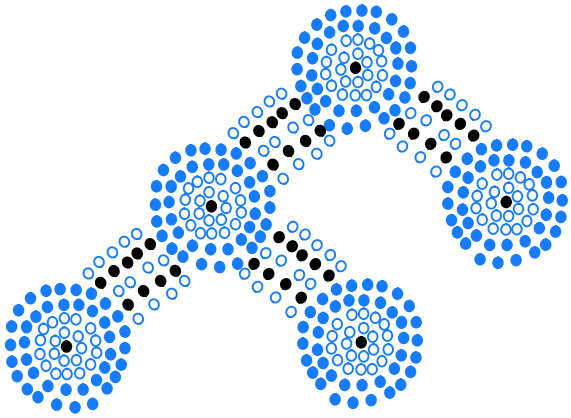
Chapter 8 Trees



Contents

8.1 General Trees 308
8.1.1 Tree Definitions and Properties 309
8.1.2 The Tree Abstract Data Type 312
8.1.3 Computing Depth and Height 314
8.2 Binary Trees 317
8.2.1 The Binary Tree Abstract Data Type 319
8.2.2 Properties of Binary Trees 321
8.3 Implementing Trees 323
8.3.1 Linked Structure for Binary Trees 323
8.3.2 Array-Based Representation of a Binary Tree 331
8.3.3 Linked Structure for General Trees 333
8.4 Tree Traversal Algorithms 334
8.4.1 Preorder and Postorder Traversals of General Trees 334
8.4.2 Breadth-First Tree Traversal 336
8.4.3 Inorder Traversal of a Binary Tree 337
8.4.4 Implementing Tree Traversals in Java 339
8.4.5 Applications of Tree Traversals 343
8.4.6 Euler Tours 348
8.5 Exercises 350

www.EBooksWorld.ir

8.1 General Trees

Productivity experts say that breakthroughs come by thinking "nonlinearly." In this chapter, we will discuss one of the most important nonlinear data structures in computing—***trees***. Tree structures are indeed a breakthrough in data organization, for they allow us to implement a host of algorithms much faster than when using linear data structures, such as arrays or linked lists. Trees also provide a natural organization for data, and consequently have become ubiquitous structures in file systems, graphical user interfaces, databases, websites, and many other computer systems.

It is not always clear what productivity experts mean by "nonlinear" thinking, but when we say that trees are "nonlinear," we are referring to an organizational relationship that is richer than the simple "before" and "after" relationships between objects in sequences. The relationships in a tree are ***hierarchical***, with some objects being "above" and some "below" others. Actually, the main terminology for tree data structures comes from family trees, with the terms "parent," "child," "ancestor," and "descendant" being the most common words used to describe relationships. We show an example of a family tree in Figure 8.1.

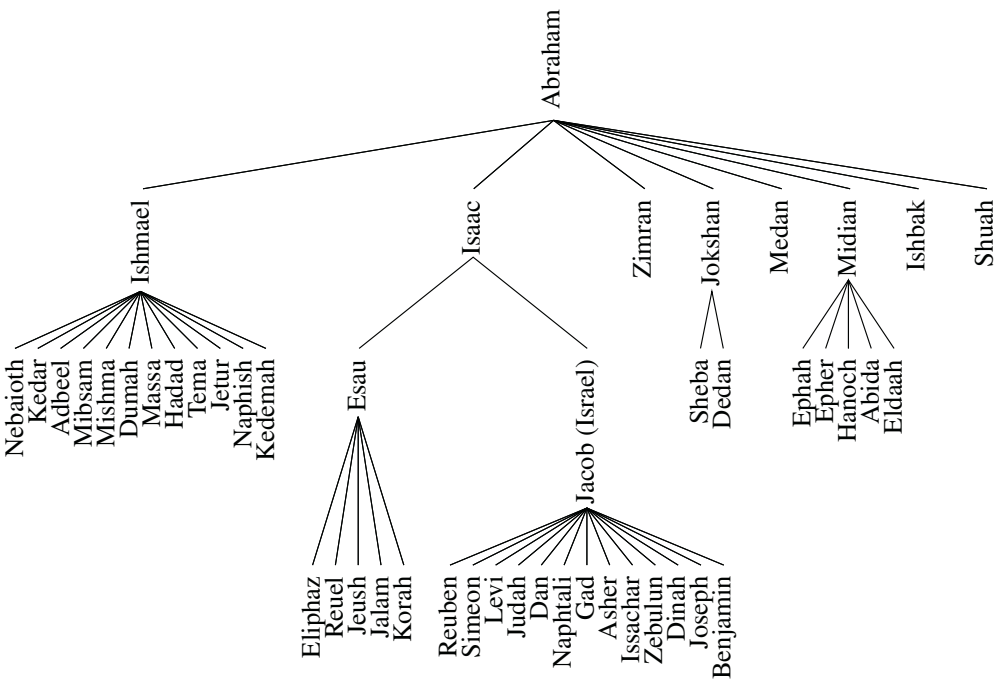


Figure 8.1: A family tree showing some descendants of Abraham, as recorded in Genesis, chapters 25–36.

www.EBooksWorld.ir

8.1.1 Tree Definitions and Properties

A ***tree*** is an abstract data type that stores elements hierarchically. With the exception of the top element, each element in a tree has a ***parent*** element and zero or more ***children*** elements. A tree is usually visualized by placing elements inside ovals or rectangles, and by drawing the connections between parents and children with straight lines. (See Figure 8.2.) We typically call the top element the ***root*** of the tree, but it is drawn as the highest element, with the other elements being connected below (just the opposite of a botanical tree).

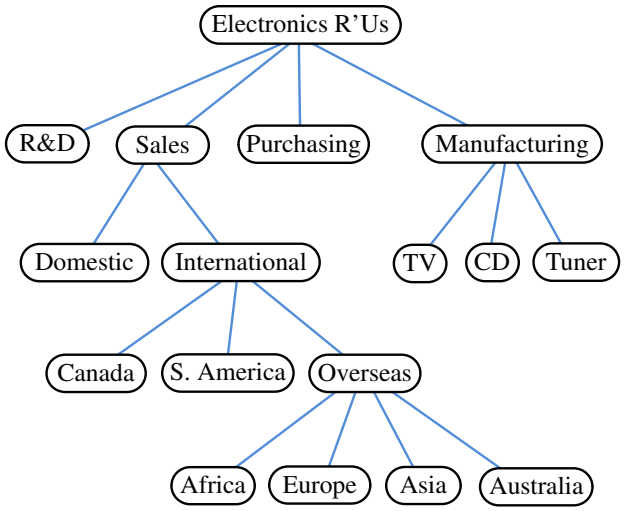


Figure 8.2: A tree with 17 nodes representing the organization of a fictitious corporation. The root stores *Electronics R'Us*. The children of the root store *R&D*, *Sales*, *Purchasing*, and *Manufacturing*. The internal nodes store *Sales*, *International*, *Overseas*, *Electronics R'Us*, and *Manufacturing*.

Formal Tree Definition

Formally, we define a ***tree*** T as a set of ***nodes*** storing elements such that the nodes have a ***parent-child*** relationship that satisfies the following properties:

- If T is nonempty, it has a special node, called the ***root*** of T, that has no parent.
- Each node v of T different from the root has a unique ***parent*** node w; every node with parent w is a ***child*** of w.

Note that according to our definition, a tree can be empty, meaning that it does not have any nodes. This convention also allows us to define a tree recursively such that a tree T is either empty or consists of a node r, called the root of T, and a (possibly empty) set of subtrees whose roots are the children of r.

www.EBooksWorld.ir

Other Node Relationships

Two nodes that are children of the same parent are ***siblings***. A node v is ***external*** if v has no children. A node v is ***internal*** if it has one or more children. External nodes are also known as ***leaves***.

Example 8.1: *In Section 5.1.4, we discussed the hierarchical relationship between files and directories in a computer's file system, although at the time we did not emphasize the nomenclature of a file system as a tree. In Figure 8.3, we revisit an earlier example. We see that the internal nodes of the tree are associated with directories and the leaves are associated with regular files. In the Unix and Linux operating systems, the root of the tree is appropriately called the "root directory," and is represented by the symbol "/."*

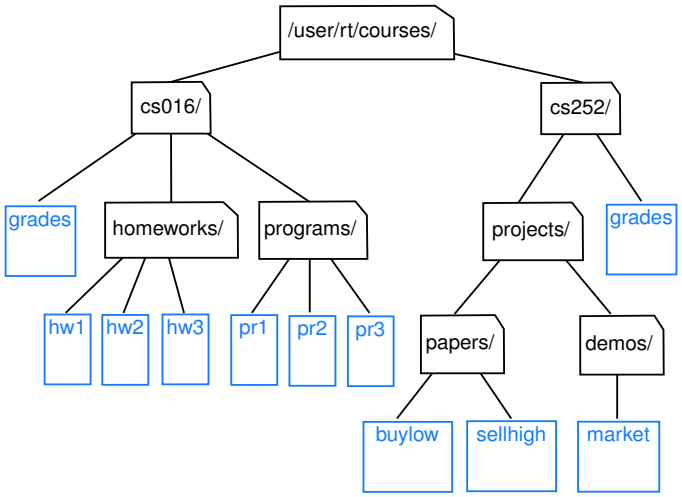


Figure 8.3: Tree representing a portion of a file system.

A node u is an ***ancestor*** of a node v if $u = v$ or u is an ancestor of the parent of v. Conversely, we say that a node v is a ***descendant*** of a node u if u is an ancestor of v. For example, in Figure 8.3, cs252/ is an ancestor of papers/, and pr3 is a descendant of cs016/. The ***subtree*** of T ***rooted*** at a node v is the tree consisting of all the descendants of v in T (including v itself). In Figure 8.3, the subtree rooted at cs016/ consists of the nodes cs016/, grades, homeworks/, programs/, hw1, hw2, hw3, pr1, pr2, and pr3.

Edges and Paths in Trees

An ***edge*** of tree T is a pair of nodes (u,v) such that u is the parent of v, or vice versa. A ***path*** of T is a sequence of nodes such that any two consecutive nodes in the sequence form an edge. For example, the tree in Figure 8.3 contains the path (cs252/, projects/, demos/, market).

www.EBooksWorld.ir

Ordered Trees

A tree is ***ordered*** if there is a meaningful linear order among the children of each node; that is, we purposefully identify the children of a node as being the first, second, third, and so on. Such an order is usually visualized by arranging siblings left to right, according to their order.

Example 8.2: *The components of a structured document, such as a book, are hierarchically organized as a tree whose internal nodes are parts, chapters, and sections, and whose leaves are paragraphs, tables, figures, and so on. (See Figure 8.4.) The root of the tree corresponds to the book itself. We could, in fact, consider expanding the tree further to show paragraphs consisting of sentences, sentences consisting of words, and words consisting of characters. Such a tree is an example of an ordered tree, because there is a well-defined order among the children of each node.*

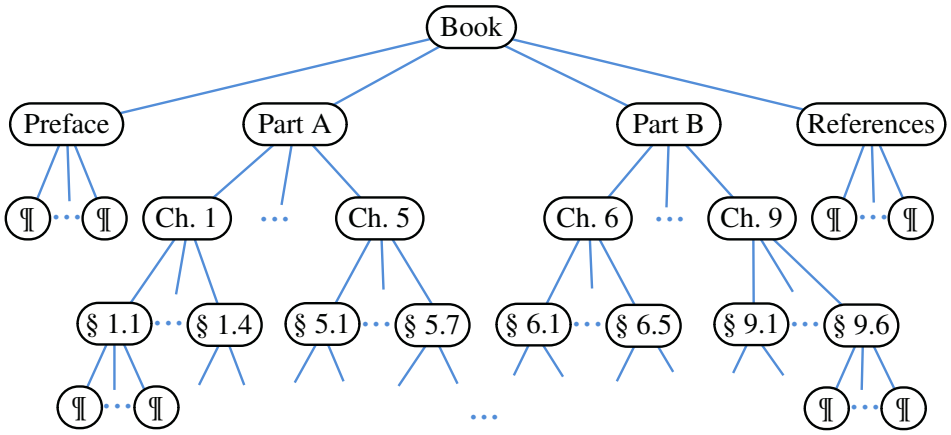


Figure 8.4: An ordered tree associated with a book.

Let's look back at the other examples of trees that we have described thus far, and consider whether the order of children is significant. A family tree that describes generational relationships, as in Figure 8.1, is often modeled as an ordered tree, with siblings ordered according to their birth.

In contrast, an organizational chart for a company, as in Figure 8.2, is typically considered an unordered tree. Likewise, when using a tree to describe an inheritance hierarchy, as in Figure 2.7, there is no particular significance to the order among the subclasses of a parent class. Finally, we consider the use of a tree in modeling a computer's file system, as in Figure 8.3. Although an operating system often displays entries of a directory in a particular order (e.g., alphabetical, chronological), such an order is not typically inherent to the file system's representation.

www.EBooksWorld.ir

8.1.2 The Tree Abstract Data Type

As we did with positional lists in Section 7.3, we define a tree ADT using the concept of a ***position*** as an abstraction for a node of a tree. An element is stored at each position, and positions satisfy parent-child relationships that define the tree structure. A position object for a tree supports the method:

getElement( ): Returns the element stored at this position.

The tree ADT then supports the following ***accessor methods***, allowing a user to navigate the various positions of a tree T:

root( ): Returns the position of the root of the tree (or null if empty).

parent(p): Returns the position of the parent of position p (or null if p is the root).

children(p): Returns an iterable collection containing the children of position p (if any).

numChildren(p): Returns the number of children of position p.

If a tree T is ordered, then children(p) reports the children of p in order.

In addition to the above fundamental accessor methods, a tree supports the following ***query methods***:

isInternal(p): Returns true if position p has at least one child.

isExternal(p): Returns true if position p does not have any children.

isRoot(p): Returns true if position p is the root of the tree.

These methods make programming with trees easier and more readable, since we can use them in the conditionals of **if** statements and **while** loops.

Trees support a number of more general methods, unrelated to the specific structure of the tree. These incude:

size( ): Returns the number of positions (and hence elements) that are contained in the tree.

isEmpty( ): Returns true if the tree does not contain any positions (and thus no elements).

iterator( ): Returns an iterator for all elements in the tree (so that the tree itself is Iterable).

positions( ): Returns an iterable collection of all positions of the tree.

If an invalid position is sent as a parameter to any method of a tree, then an IllegalArgumentException is thrown.

We do not define any methods for creating or modifying trees at this point. We prefer to describe different tree update methods in conjunction with specific implementations of the tree interface, and specific applications of trees.

www.EBooksWorld.ir

A Tree Interface in Java

In Code Fragment 8.1, we formalize the Tree ADT by defining the Tree interface in Java. We rely upon the same definition of the Position interface as introduced for positional lists in Section 7.3.2. Note well that we declare the Tree interface to formally extend Java's Iterable interface (and we include a declaration of the required iterator( ) method).

```
1  /** An interface for a tree where nodes can have an arbitrary number of children. */
2  public interface Tree<E> extends Iterable<E> {
3    Position<E> root( );
4    Position<E> parent(Position<E> p) throws IllegalArgumentException;
5    Iterable<Position<E>> children(Position<E> p)
6                                        throws IllegalArgumentException;
7    int numChildren(Position<E> p) throws IllegalArgumentException;
8    boolean isInternal(Position<E> p) throws IllegalArgumentException;
9    boolean isExternal(Position<E> p) throws IllegalArgumentException;
10   boolean isRoot(Position<E> p) throws IllegalArgumentException;
11   int size( );
12   boolean isEmpty( );
13   Iterator<E> iterator( );
14   Iterable<Position<E>> positions( );
15 }
```

Code Fragment 8.1: Definition of the Tree interface.

An AbstractTree Base Class in Java

In Section 2.3, we discussed the role of interfaces and abstract classes in Java. While an interface is a type definition that includes public declarations of various methods, an interface cannot include definitions for any of those methods. In contrast, an ***abstract class*** may define concrete implementations for some of its methods, while leaving other abstract methods without definition.

An abstract class is designed to serve as a base class, through inheritance, for one or more concrete implementations of an interface. When some of the functionality of an interface is implemented in an abstract class, less work remains to complete a concrete implementation. The standard Java libraries include many such abstract classes, including several within the Java Collections Framework. To make their purpose clear, those classes are conventionally named beginning with the word Abstract. For example, there is an AbstractCollection class that implements some of the functionality of the Collection interface, an AbstractQueue class that implements some of the functionality of the Queue interface, and an AbstractList class that implements some of the functionality of the List interface.

www.EBooksWorld.ir

In the case of our Tree interface, we will define an AbstractTree base class, demonstrating how many tree-based algorithms can be described independently of the low-level representation of a tree data structure. In fact, if a concrete implementation provides three fundamental methods—root( ), parent(p), and children(p)—all other behaviors of the Tree interface can be derived within the AbstractTree base class.

Code Fragment 8.2 presents an initial implementation of an AbstractTree base class that provides the most trivial methods of the Tree interface. We will defer until Section 8.4 a discussion of general tree-traversal algorithms that can be used to produced the positions( ) iteration within the AbstractTree class. As with our positional list ADT in Chapter 7, the iteration of *positions* of the tree can easily be adapted to produce an iteration of the *elements* of a tree, or even to determine the size of a tree (although our concrete tree implementations will provide more direct means for reporting the size).

```
1  /** An abstract base class providing some functionality of the Tree interface. */
2  public abstract class AbstractTree<E> implements Tree<E> {
3    public boolean isInternal(Position<E> p) { return numChildren(p) > 0; }
4    public boolean isExternal(Position<E> p) { return numChildren(p) == 0; }
5    public boolean isRoot(Position<E> p) { return p == root(); }
6    public boolean isEmpty( ) { return size( ) == 0; }
7  }
```

Code Fragment 8.2: An initial implementation of the AbstractTree base class. (We add additional functionality to this class as the chapter continues.)

8.1.3 Computing Depth and Height

Let p be a position within tree T. The ***depth*** of p is the number of ancestors of p, other than p itself. For example, in the tree of Figure 8.2, the node storing *International* has depth 2. Note that this definition implies that the depth of the root of T is 0. The depth of p can also be recursively defined as follows:

- If p is the root, then the depth of p is 0.
- Otherwise, the depth of p is one plus the depth of the parent of p.

Based on this definition, we present a simple recursive algorithm, depth, in Code Fragment 8.3, for computing the depth of a position p in tree T. This method calls itself recursively on the parent of p, and adds 1 to the value returned.

The running time of depth(p) for position p is $O(d_p + 1)$, where d_p denotes the depth of p in the tree, because the algorithm performs a constant-time recursive step for each ancestor of p. Thus, algorithm depth(p) runs in $O(n)$ worst-case time, where n is the total number of positions of T, because a position of T may have depth $n-1$ if all nodes form a single branch. Although such a running time is a function of the input size, it is more informative to characterize the running time in terms of the parameter d_p, as this parameter may be much smaller than n.

www.EBooksWorld.ir

```
1  /** Returns the number of levels separating Position p from the root. */
2  public int depth(Position<E> p) {
3    if (isRoot(p))
4      return 0;
5    else
6      return 1 + depth(parent(p));
7  }
```

Code Fragment 8.3: Method depth, as implemented within the AbstractTree class.

Height

We next define the ***height*** of a tree to be equal to the maximum of the depths of its positions (or zero, if the tree is empty). For example, the tree of Figure 8.2 has height 4, as the node storing *Africa* (and its siblings) has depth 4. It is easy to see that the position with maximum depth must be a leaf.

In Code Fragment 8.4, we present a method that computes the height of a tree based on this definition. Unfortunately, such an approach is not very efficient, and so name the algorithm heightBad and declare it as a private method of the AbstractTree class (so that it cannot be used by others).

```
1  /** Returns the height of the tree. */
2  private int heightBad( ) {                    // works, but quadratic worst-case time
3    int h = 0;
4    for (Position<E> p : positions( ))
5      if (isExternal(p))                        // only consider leaf positions
6        h = Math.max(h, depth(p));
7    return h;
8  }
```

Code Fragment 8.4: Method heightBad of the AbstractTree class. Note that this method calls the depth method from Code Fragment 8.3.

Although we have not yet defined the positions( ) method, we will see that it can be implemented such that the entire iteration runs in $O(n)$ time, where n is the number of positions of T. Because heightBad calls algorithm depth(p) on each leaf of T, its running time is $O(n + \sum_{p \in L}(d_p + 1))$, where L is the set of leaf positions of T. In the worst case, the sum $\sum_{p \in L}(d_p + 1)$ is proportional to n^2. (See Exercise C-8.31.) Thus, algorithm heightBad runs in $O(n^2)$ worst-case time.

We can compute the height of a tree more efficiently, in $O(n)$ worst-case time, by considering a recursive definition. To do this, we will parameterize a function based on a position within the tree, and calculate the height of the subtree rooted at that position. Formally, we define the ***height*** of a position p in a tree T as follows:

- If p is a leaf, then the height of p is 0.
- Otherwise, the height of p is one more than the maximum of the heights of p's children.

www.EBooksWorld.ir

The following proposition relates our original definition of the height of a tree to the height of the *root* position using this recursive formula.

Proposition 8.3: *The height of the root of a nonempty tree T, according to the recursive definition, equals the maximum depth among all leaves of tree T.*

We leave the justification of this proposition as Exercise R-8.3.

An implementation of a recursive algorithm to compute the height of a subtree rooted at a given position p is presented in Code Fragment 8.5. The overall height of a nonempty tree can be computed by sending the root of the tree as a parameter.

```
1  /** Returns the height of the subtree rooted at Position p. */
2  public int height(Position<E> p) {
3    int h = 0;                                    // base case if p is external
4    for (Position<E> c : children(p))
5      h = Math.max(h, 1 + height(c));
6    return h;
7  }
```

Code Fragment 8.5: Method height for computing the height of a subtree rooted at a position p of an AbstractTree.

It is important to understand why method height is more efficient than method heightBad. The algorithm is recursive, and it progresses in a top-down fashion. If the method is initially called on the root of T, it will eventually be called once for each position of T. This is because the root eventually invokes the recursion on each of its children, which in turn invokes the recursion on each of their children, and so on.

We can determine the running time of the recursive height algorithm by summing, over all the positions, the amount of time spent on the nonrecursive part of each call. (Review Section 5.2 for analyses of recursive processes.) In our implementation, there is a constant amount of work per position, plus the overhead of computing the maximum over the iteration of children. Although we do not yet have a concrete implementation of children(p), we assume that such an iteration is executed in $O(c_p+1)$ time, where c_p denotes the number of children of p. Algorithm height(p) spends $O(c_p+1)$ time at each position p to compute the maximum, and its overall running time is $O(\sum_p(c_p+1)) = O(n+\sum_p c_p)$. In order to complete the analysis, we make use of the following property.

Proposition 8.4: *Let T be a tree with n positions, and let c_p denote the number of children of a position p of T. Then, summing over the positions of T, $\sum_p c_p = n-1$.*

Justification: Each position of T, with the exception of the root, is a child of another position, and thus contributes one unit to the above sum. ■

By Proposition 8.4, the running time of algorithm height, when called on the root of T, is $O(n)$, where n is the number of positions of T.

www.EBooksWorld.ir